

数论

yx-12243

February 18 ~ 19, 2021

1 整除和最大公约数

- 整除理论
- 最大公约数

■ 同余

2 素数

- 素数和逆

Definition 1.1 (Divisible)

对于两个整数 a, b ($b \neq 0$), 如果存在整数 q 使得 $a = b \cdot q$, 则称 b 整除 a , a 被 b 整除, 记作 $b \mid a$ 。

Definition 1.1 (Divisible)

对于两个整数 a, b ($b \neq 0$), 如果存在整数 q 使得 $a = b \cdot q$, 则称 b 整除 a , a 被 b 整除, 记作 $b \mid a$ 。

当然, 并不是所有时候都能找到这样的 q , 因此对一般的 a 可以定义:

Definition 1.1 (Divisible)

对于两个整数 a, b ($b \neq 0$), 如果存在整数 q 使得 $a = b \cdot q$, 则称 b 整除 a , a 被 b 整除, 记作 $b \mid a$ 。

当然, 并不是所有时候都能找到这样的 q , 因此对一般的 a 可以定义:

Definition 1.2 (Division with remainder)

对于两个整数 a, b ($b \neq 0$), 存在唯一的整数对 (q, r) 满足 $a = b \cdot q + r$ 且 $0 \leq r < |b|$ 。称 q 为 a 除以 b 的商, r 为 a 除以 b 的余数 (模)。用 $a \div b = q \dots r$ 表示。

如果用 $\lfloor x \rfloor$ 表示不超过 x 的最大整数， $\lceil x \rceil$ 表示不小于 x 的最小整数，则带余除法和取整之间由如下关系：

如果用 $\lfloor x \rfloor$ 表示不超过 x 的最大整数, $\lceil x \rceil$ 表示不小于 x 的最小整数, 则带余除法和取整之间由如下关系:

Theorem 1.1

若 $a \div b = q \dots r$, 则 $q = \left\lfloor \frac{a}{b} \right\rfloor$, $r = a - b \cdot q = a - b \left\lfloor \frac{a}{b} \right\rfloor$
 $\stackrel{\text{def}}{=} a \bmod b$.

如果用 $\lfloor x \rfloor$ 表示不超过 x 的最大整数, $\lceil x \rceil$ 表示不小于 x 的最小整数, 则带余除法和取整之间由如下关系:

Theorem 1.1

若 $a \div b = q \dots r$, 则 $q = \left\lfloor \frac{a}{b} \right\rfloor$, $r = a - b \cdot q = a - b \left\lfloor \frac{a}{b} \right\rfloor$
 $\stackrel{\text{def}}{=} a \bmod b$ 。

(注: 在大多数语言是一般用 $a \% b$ 来表示 $a \bmod b$, 但注意当 b 是负数时不同的语言具有不同的行为, 如 *C++* 向零取整, *Python* 向下取整。因此实际使用时应避免 b 为负数时的情形)

Theorem 1.2

若 m 是正整数, x 是实数, 则 $\left\lfloor \frac{x}{m} \right\rfloor = \left\lfloor \frac{\lfloor x \rfloor}{m} \right\rfloor$ 。

Theorem 1.2

若 m 是正整数, x 是实数, 则 $\left\lfloor \frac{x}{m} \right\rfloor = \left\lfloor \frac{\lfloor x \rfloor}{m} \right\rfloor$ 。

Corollary 1.1

若 b, c 是正整数, a 是整数, 则 $\left\lfloor \frac{a}{bc} \right\rfloor = \left\lfloor \frac{\left\lfloor \frac{a}{b} \right\rfloor}{c} \right\rfloor$ 。

在遇到上下取整的问题时，下面一些不等式可以帮助我们进行转化。首先是两个最基本的定理：

在遇到上下取整的问题时，下面一些不等式可以帮助我们进行转化。首先是两个最基本的定理：

Theorem 1.3

设 n 是整数， x 是实数，则

$$n \leq \lfloor x \rfloor \text{ 当且仅当 } n \leq x, \quad n \geq \lceil x \rceil \text{ 当且仅当 } n \geq x.$$

在遇到上下取整的问题时，下面一些不等式可以帮助我们进行转化。首先是两个最基本的定理：

Theorem 1.3

设 n 是整数， x 是实数，则

$$n \leq \lfloor x \rfloor \text{ 当且仅当 } n \leq x, \quad n \geq \lceil x \rceil \text{ 当且仅当 } n \geq x.$$

而对于像 $n \geq \lfloor x \rfloor$ 之类的不等式，首先可以利用整数的离散性得 $n + 1 > \lfloor x \rfloor$ ，即 $n + 1 \leq \lceil x \rceil$ 的否命题，然后得 $n > x - 1$ 。

在遇到上下取整的问题时，下面一些不等式可以帮助我们进行转化。首先是两个最基本的定理：

Theorem 1.3

设 n 是整数， x 是实数，则

$$n \leq \lfloor x \rfloor \text{ 当且仅当 } n \leq x, \quad n \geq \lceil x \rceil \text{ 当且仅当 } n \geq x.$$

而对于像 $n \geq \lfloor x \rfloor$ 之类的不等式，首先可以利用整数的离散性得 $n+1 > \lfloor x \rfloor$ ，即 $n+1 \leq \lceil x \rceil$ 的否命题，然后得 $n > x-1$ 。

Corollary 1.2

设 n 是整数， x 是实数，则

$$n \leq \lfloor x \rfloor \text{ 当且仅当 } n \leq x, \quad n \geq \lfloor x \rfloor \text{ 当且仅当 } n > x-1,$$

$$n \leq \lceil x \rceil \text{ 当且仅当 } n < x+1, \quad n \geq \lceil x \rceil \text{ 当且仅当 } n \geq x.$$

Corollary 1.3

在 *Corollary 1.2* 中令 $n = \lfloor x \rfloor$ 与 $\lceil x \rceil$, 得

$$x - 1 < \lfloor x \rfloor \leq x, \quad x \leq \lceil x \rceil < x + 1.$$

Example 1

对于给定的正整数 n, m , 求所有正整数 x 使得 $\left\lfloor \frac{n}{x} \right\rfloor = m$ 。

Example 1

对于给定的正整数 n, m , 求所有正整数 x 使得 $\left\lfloor \frac{n}{x} \right\rfloor = m$ 。

Solution

$$\left\lfloor \frac{n}{x} \right\rfloor = m \Leftrightarrow m \leq \left\lfloor \frac{n}{x} \right\rfloor \text{ 且 } m \geq \left\lfloor \frac{n}{x} \right\rfloor。$$

Example 1

对于给定的正整数 n, m , 求所有正整数 x 使得 $\left\lfloor \frac{n}{x} \right\rfloor = m$ 。

Solution

$$\left\lfloor \frac{n}{x} \right\rfloor = m \Leftrightarrow m \leq \left\lfloor \frac{n}{x} \right\rfloor \text{ 且 } m \geq \left\lfloor \frac{n}{x} \right\rfloor。$$

由 Corollary 1.2 得 $\frac{n}{x} - 1 < m \leq \frac{n}{x}$, 即 $\frac{n}{m+1} < x \leq \frac{n}{m}$ 。

Example 1

对于给定的正整数 n, m , 求所有正整数 x 使得 $\left\lfloor \frac{n}{x} \right\rfloor = m$ 。

Solution

$$\left\lfloor \frac{n}{x} \right\rfloor = m \Leftrightarrow m \leq \left\lfloor \frac{n}{x} \right\rfloor \text{ 且 } m \geq \left\lfloor \frac{n}{x} \right\rfloor。$$

由 Corollary 1.2 得 $\frac{n}{x} - 1 < m \leq \frac{n}{x}$, 即 $\frac{n}{m+1} < x \leq \frac{n}{m}$ 。

再利用 Corollary 1.2 得 $\left\lfloor \frac{n}{m+1} \right\rfloor + 1 \leq x \leq \left\lfloor \frac{n}{m} \right\rfloor$, 即 x 可以

取 $\left\lfloor \frac{n}{m+1} \right\rfloor + 1 \sim \left\lfloor \frac{n}{m} \right\rfloor$ 之间的所有整数 (当然有可能是空集)。

我们用 $\mathcal{S}(n)$ 表示所有正整数 m , 使得存在至少一个 $x \in \mathbb{N}^*$ 满足 $\left\lfloor \frac{n}{x} \right\rfloor = m$ 。由定义知

$$\mathcal{S}(n) = \left\{ \left\lfloor \frac{n}{i} \right\rfloor \mid 1 \leq i \leq n \right\}$$

我们用 $\mathcal{S}(n)$ 表示所有正整数 m , 使得存在至少一个 $x \in \mathbb{N}^*$ 满足 $\left\lfloor \frac{n}{x} \right\rfloor = m$ 。由定义知

$$\mathcal{S}(n) = \left\{ \left\lfloor \frac{n}{i} \right\rfloor \mid 1 \leq i \leq n \right\}$$

Theorem 1.4

设 $B = \lfloor \sqrt{n} \rfloor$, 则

$$\mathcal{S}(n) = \{1, 2, \dots, B\} \cup \left\{ \left\lfloor \frac{n}{i} \right\rfloor \mid 1 \leq i \leq B \right\}$$

$$\text{且 } |\mathcal{S}(n)| = \lfloor \sqrt{4n+1} \rfloor - 1 = 2\sqrt{n} + O(1) = O(\sqrt{n}).$$

我们用 $\mathcal{S}(n)$ 表示所有正整数 m , 使得存在至少一个 $x \in \mathbb{N}^*$ 满足 $\left\lfloor \frac{n}{x} \right\rfloor = m$ 。由定义知

$$\mathcal{S}(n) = \left\{ \left\lfloor \frac{n}{i} \right\rfloor \mid 1 \leq i \leq n \right\}$$

Theorem 1.4

设 $B = \lfloor \sqrt{n} \rfloor$, 则

$$\mathcal{S}(n) = \{1, 2, \dots, B\} \cup \left\{ \left\lfloor \frac{n}{i} \right\rfloor \mid 1 \leq i \leq B \right\}$$

$$\text{且 } |\mathcal{S}(n)| = \lfloor \sqrt{4n+1} \rfloor - 1 = 2\sqrt{n} + O(1) = O(\sqrt{n}).$$

我们把利用 $|\mathcal{S}(n)| = O(\sqrt{n})$ 来解决一些问题的方法称为**整除分块**。

Example 2

给定正整数 n , 求 $\sum_{i=1}^n i \cdot \left\lfloor \frac{n}{i} \right\rfloor$ 。
 $n \leq 10^{12}$ 。

Example 2

给定正整数 n , 求 $\sum_{i=1}^n i \cdot \left\lfloor \frac{n}{i} \right\rfloor$ 。

$n \leq 10^{12}$ 。

解决这类问题的主要思路就是, 枚举 $\mathcal{S}(n)$ 中的元素 m , 然后找到满足 $\left\lfloor \frac{n}{i} \right\rfloor = m$ 的 i 的范围 $[l, r]$, 计算 $\sum_{i=l}^r i \cdot m$ 。

整除分块的实现是由必要提及的。因为整除分块中涉及大量除法，其中常数因子的影响是不可忽略的 (甚至可以成为运行时间的瓶颈)。

整除分块的实现是由必要提及的。因为整除分块中涉及大量除法，其中常数因子的影响是不可忽略的 (甚至可以成为运行时间的瓶颈)。

网上大多数的整除分块在一遍过程中需要用到 $4\sqrt{n}$ 甚至 $6\sqrt{n}$ 次 64 位整数除法。而下面的实现对于一个固定的 n ，只需要 $\sqrt{n}$ 次除法的预处理，接下来的过程中不需要进行任何除法操作。

整除分块的实现是由必要提及的。因为整除分块中涉及大量除法，其中常数因子的影响是不可忽略的 (甚至可以成为运行时间的瓶颈)。

网上大多数的整除分块在一遍过程中需要用到 $4\sqrt{n}$ 甚至 $6\sqrt{n}$ 次 64 位整数除法。而下面的实现对于一个固定的 n ，只需要 $\sqrt{n}$ 次除法的预处理，接下来的过程中不需要进行任何除法操作。

(注：经过本人测试，在大多数计算机中，内存访问是比 64 位整数除法要快的)

整除分块的实现是由必要提及的。因为整除分块中涉及大量除法，其中常数因子的影响是不可忽略的 (甚至可以成为运行时间的瓶颈)。

网上大多数的整除分块在一遍过程中需要用到 $4\sqrt{n}$ 甚至 $6\sqrt{n}$ 次 64 位整数除法。而下面的实现对于一个固定的 n ，只需要 $\sqrt{n}$ 次除法的预处理，接下来的过程中不需要进行任何除法操作。

(注：经过本人测试，在大多数计算机中，内存访问是比 64 位整数除法要快的)

```
1  srn = sqrt(n), cnt = srn * 2 - (srn * (srn + 1ll) > n);  
2  for (int i = 1; i <= srn; ++i) {  
3      v[i] = i, v[cnt - i + 1] = n / i;  
4  }  
5  for (int i = 1, j = cnt; i <= cnt; ++i, --j) {  
6      // [n / x] = v[i], range of x = (v[j - 1], v[j])  
7  }
```

整除分块的实现是由必要提及的。因为整除分块中涉及大量除法，其中常数因子的影响是不可忽略的 (甚至可以成为运行时间的瓶颈)。

网上大多数的整除分块在一遍过程中需要用到 $4\sqrt{n}$ 甚至 $6\sqrt{n}$ 次 64 位整数除法。而下面的实现对于一个固定的 n ，只需要 $\sqrt{n}$ 次除法的预处理，接下来的过程中不需要进行任何除法操作。

(注：经过本人测试，在大多数计算机中，内存访问是比 64 位整数除法要快的)

```
1  srn = sqrt(n), cnt = srn * 2 - (srn * (srn + 1ll) > n);  
2  for (int i = 1; i <= srn; ++i) {  
3      v[i] = i, v[cnt - i + 1] = n / i;  
4  }  
5  for (int i = 1, j = cnt; i <= cnt; ++i, --j) {  
6      // [n / x] = v[i], range of x = (v[j - 1], v[j])  
7  }
```

整除分块是数论中的一个非常基础且重要的技巧，在后面的各种问题中，将会多次看到它的应用。

Definition 2.1 (Divisors and multiples)

若 $b \mid a$, 则称 b 是 a 的因数 (约数), a 是 b 的倍数。



Definition 2.1 (Divisors and multiples)

若 $b \mid a$, 则称 b 是 a 的因数 (约数), a 是 b 的倍数。

对于一个正整数 n , 我们一般用 $\mathcal{D}(n)$ 表示 n 的正因数集合, $d(n) = |\mathcal{D}(n)|$ 表示 n 的正因数数量。

Definition 2.1 (Divisors and multiples)

若 $b \mid a$, 则称 b 是 a 的**因数** (约数), a 是 b 的**倍数**。

对于一个正整数 n , 我们一般用 $\mathcal{D}(n)$ 表示 n 的正因数集合, $d(n) = |\mathcal{D}(n)|$ 表示 n 的正因数数量。

注意到 n 的正因数一定不超过 n , 因此 $\mathcal{D}(n)$ 是有限集。其中最小元为 1, 最大元为 n 。

Definition 2.1 (Divisors and multiples)

若 $b \mid a$, 则称 b 是 a 的**因数** (约数), a 是 b 的**倍数**。

对于一个正整数 n , 我们一般用 $\mathcal{D}(n)$ 表示 n 的正因数集合, $d(n) = |\mathcal{D}(n)|$ 表示 n 的正因数数量。

注意到 n 的正因数一定不超过 n , 因此 $\mathcal{D}(n)$ 是有限集。其中最小元为 1, 最大元为 n 。

接下来我们引入一个非常重要的概念：**最大公约数**。

Definition 2.1 (Divisors and multiples)

若 $b \mid a$, 则称 b 是 a 的**因数** (约数), a 是 b 的**倍数**。

对于一个正整数 n , 我们一般用 $\mathcal{D}(n)$ 表示 n 的正因数集合, $d(n) = |\mathcal{D}(n)|$ 表示 n 的正因数数量。

注意到 n 的正因数一定不超过 n , 因此 $\mathcal{D}(n)$ 是有限集。其中最小元为 1, 最大元为 n 。

接下来我们引入一个非常重要的概念：**最大公约数**。

Definition 2.2 (Greatest common divisor)

对于两个不全为零的整数 a, b , 称满足 $d \mid a$ 且 $d \mid b$ 的最大整数 d 为 a, b 的**最大公约数**, 记为 $\gcd(a, b)$, 在不引起歧义的情况下可简写为 (a, b) 。

特别地, 若 $a = b = 0$, 则定义 $\gcd(0, 0) = 0$ 。

Definition 2.3 (Least common multiple)

同样，对两个非零整数 a, b ，称满足 $a \mid m$ 且 $b \mid m$ 的最小正整数为 a, b 的最小公倍数，记为 $\text{lcm}(a, b)$ ，在不引起歧义的情况下可简写为 $[a, b]$ 。

特别地，定义 $\text{lcm}(a, 0) = \text{lcm}(0, b) = 0$ 。

Definition 2.3 (Least common multiple)

同样, 对两个非零整数 a, b , 称满足 $a \mid m$ 且 $b \mid m$ 的最小正整数为 a, b 的最小公倍数, 记为 $\text{lcm}(a, b)$, 在不引起歧义的情况下可简写为 $[a, b]$ 。

特别地, 定义 $\text{lcm}(a, 0) = \text{lcm}(0, b) = 0$ 。

类似可以定义多个数的最大公约数、最小公倍数。



最大公约数和最小公倍数有如下性质：

最大公约数和最小公倍数有如下性质：

Theorem 2.1

- $\gcd(\gcd(a, b), c) = \gcd(a, b, c)$ 。



最大公约数和最小公倍数有如下性质：

Theorem 2.1

- $\gcd(\gcd(a, b), c) = \gcd(a, b, c)$ 。
- $\mathcal{D}(a) \cap \mathcal{D}(b) = \mathcal{D}(\gcd(a, b))$ 。



最大公约数和最小公倍数有如下性质：

Theorem 2.1

- $\gcd(\gcd(a, b), c) = \gcd(a, b, c)$ 。
- $\mathcal{D}(a) \cap \mathcal{D}(b) = \mathcal{D}(\gcd(a, b))$ 。
- $\gcd(a^n, b^n) = \gcd(a, b)^n$ 。



最大公约数和最小公倍数有如下性质：

Theorem 2.1

- $\gcd(\gcd(a, b), c) = \gcd(a, b, c)$ 。
- $\mathcal{D}(a) \cap \mathcal{D}(b) = \mathcal{D}(\gcd(a, b))$ 。
- $\gcd(a^n, b^n) = \gcd(a, b)^n$ 。
- $\gcd(ma, mb) = m \gcd(a, b)$ 。

最大公约数和最小公倍数有如下性质：

Theorem 2.1

- $\gcd(\gcd(a, b), c) = \gcd(a, b, c)$ 。
- $\mathcal{D}(a) \cap \mathcal{D}(b) = \mathcal{D}(\gcd(a, b))$ 。
- $\gcd(a^n, b^n) = \gcd(a, b)^n$ 。
- $\gcd(ma, mb) = m \gcd(a, b)$ 。
- $\text{lcm}(\text{lcm}(a, b), c) = \text{lcm}(a, b, c)$ 。

最大公约数和最小公倍数有如下性质：

Theorem 2.1

- $\gcd(\gcd(a, b), c) = \gcd(a, b, c)$ 。
- $\mathcal{D}(a) \cap \mathcal{D}(b) = \mathcal{D}(\gcd(a, b))$ 。
- $\gcd(a^n, b^n) = \gcd(a, b)^n$ 。
- $\gcd(ma, mb) = m \gcd(a, b)$ 。
- $\text{lcm}(\text{lcm}(a, b), c) = \text{lcm}(a, b, c)$ 。
- $\gcd(a, b) \text{lcm}(a, b) = ab$ 。

最大公约数和最小公倍数有如下性质：

Theorem 2.1

- $\gcd(\gcd(a, b), c) = \gcd(a, b, c)$ 。
- $\mathcal{D}(a) \cap \mathcal{D}(b) = \mathcal{D}(\gcd(a, b))$ 。
- $\gcd(a^n, b^n) = \gcd(a, b)^n$ 。
- $\gcd(ma, mb) = m \gcd(a, b)$ 。
- $\text{lcm}(\text{lcm}(a, b), c) = \text{lcm}(a, b, c)$ 。
- $\gcd(a, b) \text{lcm}(a, b) = ab$ 。
- $\mathcal{D}(a) \cup \mathcal{D}(b) \subseteq \mathcal{D}(\text{lcm}(a, b))$ 。



最大公约数和最小公倍数有如下性质：

Theorem 2.1

- $\gcd(\gcd(a, b), c) = \gcd(a, b, c)$ 。
- $\mathcal{D}(a) \cap \mathcal{D}(b) = \mathcal{D}(\gcd(a, b))$ 。
- $\gcd(a^n, b^n) = \gcd(a, b)^n$ 。
- $\gcd(ma, mb) = m \gcd(a, b)$ 。
- $\text{lcm}(\text{lcm}(a, b), c) = \text{lcm}(a, b, c)$ 。
- $\gcd(a, b) \text{lcm}(a, b) = ab$ 。
- $\mathcal{D}(a) \cup \mathcal{D}(b) \subseteq \mathcal{D}(\text{lcm}(a, b))$ 。
- $a \mid b \Leftrightarrow \gcd(a, b) = a \Leftrightarrow \text{lcm}(a, b) = b$ 。



Theorem 2.2 (Bézout)

$\gcd(a, b)$ 是能用 $ax + by$ ($x, y \in \mathbb{Z}$) 表示的数中最小的正整数。



Theorem 2.2 (Bézout)

$\gcd(a, b)$ 是能用 $ax + by$ ($x, y \in \mathbb{Z}$) 表示的数中最小的正整数。

Definition 2.4 (Coprime)

特别地, 我们称 a, b **互素**, 如果 $\gcd(a, b) = 1$, 亦可记作 $a \perp b$ 。

Theorem 2.2 (Bézout)

$\gcd(a, b)$ 是能用 $ax + by$ ($x, y \in \mathbb{Z}$) 表示的数中最小的正整数。

Definition 2.4 (Coprime)

特别地, 我们称 a, b 互素, 如果 $\gcd(a, b) = 1$, 亦可记作 $a \perp b$ 。

Corollary 2.1

a, b 互素的充分必要条件是, 存在整数 x, y 使得 $ax + by = 1$ 。



最大公约数有如下性质：

Theorem 2.3

对于任意整数 a, b ，有 $\gcd(a, b) = \gcd(b, a - b)$ 。

从而对于 $a > b > 0$ ，有 $\gcd(a, b) = \gcd(b, a \bmod b)$ 。



最大公约数有如下性质：

Theorem 2.3

对于任意整数 a, b ，有 $\gcd(a, b) = \gcd(b, a - b)$ 。

从而对于 $a > b > 0$ ，有 $\gcd(a, b) = \gcd(b, a \bmod b)$ 。

考虑通过上述过程递归计算两个数的最大公约数，直到 $b = 0$ 。



最大公约数有如下性质：

Theorem 2.3

对于任意整数 a, b ，有 $\gcd(a, b) = \gcd(b, a - b)$ 。

从而对于 $a > b > 0$ ，有 $\gcd(a, b) = \gcd(b, a \bmod b)$ 。

考虑通过上述过程递归计算两个数的最大公约数，直到 $b = 0$ 。

注意到对于正整数 $a > b$ ，始终有 $a \bmod b < \frac{a}{2}$ ，因此上述递归的总步数不会超过 $2 \log \min \{a, b\}$ 。



最大公约数有如下性质：

Theorem 2.3

对于任意整数 a, b ，有 $\gcd(a, b) = \gcd(b, a - b)$ 。

从而对于 $a > b > 0$ ，有 $\gcd(a, b) = \gcd(b, a \bmod b)$ 。

考虑通过上述过程递归计算两个数的最大公约数，直到 $b = 0$ 。

注意到对于正整数 $a > b$ ，始终有 $a \bmod b < \frac{a}{2}$ ，因此上述递归的总步数不会超过 $2 \log \min \{a, b\}$ 。

这就是所谓的 **Euclid 算法**。

对其进行更细致地分析可知，Euclid 算法递归的一个更加精确的上界是 $\log_{\phi} \min \{a, b\} + O(1)$ ，其中 $\phi = \frac{1 + \sqrt{5}}{2}$ 为黄金分割数。

对其进行更细致地分析可知, Euclid 算法递归的一个更加精确的上界是 $\log_{\phi} \min \{a, b\} + O(1)$, 其中 $\phi = \frac{1 + \sqrt{5}}{2}$ 为黄金分割数。

同时, 设 $d = \gcd(a, b)$, 则有算法过程可知计算 $\gcd(a, b)$ 的时间和计算 $\gcd\left(\frac{a}{d}, \frac{b}{d}\right)$ 的时间是相同的, 因此有

Theorem 2.4

用 Euclid 算法计算两个正整数 a, b 的最大公约数的递归次数不超过 $\log_{\phi} \frac{\min \{a, b\}}{\gcd(a, b)} + O(1)$, 时间复杂度为 $O\left(\log \frac{\min \{a, b\}}{\gcd(a, b)}\right)$ 。

利用对数的性质 ($\log(ab) = \log a + \log b$), 可得

Corollary 2.2

用 *Euclid* 算法计算 n 个正整数 $a_1, a_2, \dots, a_n$ 的最大公约数的时间复杂度为 $O\left(\log \frac{\min\{a_1, a_2, \dots, a_n\}}{\gcd(a_1, a_2, \dots, a_n)}\right)$ 。

对于较大的整数，由于 Euclid 算法涉及到模运算，因此实际效率不是很高。从而，我们有下面的 Stein 算法来代替。

对于较大的整数，由于 Euclid 算法涉及到模运算，因此实际效率不是很高。从而，我们有下面的 Stein 算法来代替。

考虑计算 $\gcd(a, b)$ ($a > b$)，我们根据 a, b 的奇偶性分为四种情形：

对于较大的整数，由于 Euclid 算法涉及到模运算，因此实际效率不是很高。从而，我们有下面的 Stein 算法来代替。

考虑计算 $\gcd(a, b)$ ($a > b$)，我们根据 a, b 的奇偶性分为四种情形：

1 若 $2 \mid a$ 且 $2 \mid b$ ，则 $\gcd(a, b) = 2 \gcd\left(\frac{a}{2}, \frac{b}{2}\right)$ 。

对于较大的整数，由于 Euclid 算法涉及到模运算，因此实际效率不是很高。从而，我们有下面的 Stein 算法来代替。

考虑计算 $\gcd(a, b)$ ($a > b$)，我们根据 a, b 的奇偶性分为四种情形：

- 1 若 $2 \mid a$ 且 $2 \mid b$ ，则 $\gcd(a, b) = 2 \gcd\left(\frac{a}{2}, \frac{b}{2}\right)$ 。
- 2 若 $2 \mid a$ 且 $2 \nmid b$ ，则 $\gcd(a, b) = \gcd\left(\frac{a}{2}, b\right)$ 。

对于较大的整数，由于 Euclid 算法涉及到模运算，因此实际效率不是很高。从而，我们有下面的 Stein 算法来代替。

考虑计算 $\gcd(a, b)$ ($a > b$)，我们根据 a, b 的奇偶性分为四种情形：

- 1 若 $2 \mid a$ 且 $2 \mid b$ ，则 $\gcd(a, b) = 2 \gcd\left(\frac{a}{2}, \frac{b}{2}\right)$ 。
- 2 若 $2 \mid a$ 且 $2 \nmid b$ ，则 $\gcd(a, b) = \gcd\left(\frac{a}{2}, b\right)$ 。
- 3 若 $2 \nmid a$ 且 $2 \mid b$ ，则 $\gcd(a, b) = \gcd\left(a, \frac{b}{2}\right)$ 。

对于较大的整数，由于 Euclid 算法涉及到模运算，因此实际效率不是很高。从而，我们有下面的 Stein 算法来代替。

考虑计算 $\gcd(a, b)$ ($a > b$)，我们根据 a, b 的奇偶性分为四种情形：

- 1 若 $2 \mid a$ 且 $2 \mid b$ ，则 $\gcd(a, b) = 2 \gcd\left(\frac{a}{2}, \frac{b}{2}\right)$ 。
- 2 若 $2 \mid a$ 且 $2 \nmid b$ ，则 $\gcd(a, b) = \gcd\left(\frac{a}{2}, b\right)$ 。
- 3 若 $2 \nmid a$ 且 $2 \mid b$ ，则 $\gcd(a, b) = \gcd\left(a, \frac{b}{2}\right)$ 。
- 4 若 $2 \nmid a$ 且 $2 \nmid b$ ，则 $\gcd(a, b) = \gcd(a - b, b)$ 。

对于较大的整数，由于 Euclid 算法涉及到模运算，因此实际效率不是很高。从而，我们有下面的 Stein 算法来代替。

考虑计算 $\gcd(a, b)$ ($a > b$)，我们根据 a, b 的奇偶性分为四种情形：

- 1 若 $2 \mid a$ 且 $2 \mid b$ ，则 $\gcd(a, b) = 2 \gcd(\frac{a}{2}, \frac{b}{2})$ 。
- 2 若 $2 \mid a$ 且 $2 \nmid b$ ，则 $\gcd(a, b) = \gcd(\frac{a}{2}, b)$ 。
- 3 若 $2 \nmid a$ 且 $2 \mid b$ ，则 $\gcd(a, b) = \gcd(a, \frac{b}{2})$ 。
- 4 若 $2 \nmid a$ 且 $2 \nmid b$ ，则 $\gcd(a, b) = \gcd(a - b, b)$ 。

整个算法只用到了除以 2 和减法。如果使用二进制存储数，该算法具有较高的实际效率。

对于较大的整数，由于 Euclid 算法涉及到模运算，因此实际效率不是很高。从而，我们有下面的 Stein 算法来代替。

考虑计算 $\gcd(a, b)$ ($a > b$)，我们根据 a, b 的奇偶性分为四种情形：

- 1 若 $2 \mid a$ 且 $2 \mid b$ ，则 $\gcd(a, b) = 2 \gcd(\frac{a}{2}, \frac{b}{2})$ 。
- 2 若 $2 \mid a$ 且 $2 \nmid b$ ，则 $\gcd(a, b) = \gcd(\frac{a}{2}, b)$ 。
- 3 若 $2 \nmid a$ 且 $2 \mid b$ ，则 $\gcd(a, b) = \gcd(a, \frac{b}{2})$ 。
- 4 若 $2 \nmid a$ 且 $2 \nmid b$ ，则 $\gcd(a, b) = \gcd(a - b, b)$ 。

整个算法只用到了除以 2 和减法。如果使用二进制存储数，该算法具有较高的实际效率。

注意到相邻两次递归中至少有一次不是第 4 种，因此递归调用的总次数为 $O(\log a + \log b)$ 。

Bézout 定理告诉我们，存在整数 x, y 满足 $ax + by = (a, b)$ ，而扩展 **Euclid** 算法不仅能求出 $\gcd(a, b)$ 的值，还能求得这样一组 x, y 。

Bézout 定理告诉我们，存在整数 x, y 满足 $ax + by = (a, b)$ ，而扩展 **Euclid** 算法不仅能求出 $\gcd(a, b)$ 的值，还能求得这样一组 x, y 。

扩展 Euclid 算法也是一个递归过程：

Bézout 定理告诉我们，存在整数 x, y 满足 $ax + by = (a, b)$ ，而扩展 **Euclid** 算法不仅能求出 $\gcd(a, b)$ 的值，还能求得这样一组 x, y 。

扩展 Euclid 算法也是一个递归过程：

- 1 若 $b = 0$ ，则 $x = 1, y = 0$ 。

Bézout 定理告诉我们, 存在整数 x, y 满足 $ax + by = (a, b)$, 而扩展 Euclid 算法不仅能求出 $\gcd(a, b)$ 的值, 还能求得这样一组 x, y 。

扩展 Euclid 算法也是一个递归过程:

- 1 若 $b = 0$, 则 $x = 1, y = 0$ 。
- 2 否则, 设 $by' + (a \bmod b)x' = \gcd(a, b)$, 则利用 Theorem 1.1 知 $by' + \left(a - b \left\lfloor \frac{a}{b} \right\rfloor\right)x' = \gcd(a, b)$, 从而有 $ax' + b\left(y' - \left\lfloor \frac{a}{b} \right\rfloor x'\right) = \gcd(a, b)$, 即 $x = x', y = y' - \left\lfloor \frac{a}{b} \right\rfloor x'$ 。

其代码实现如下：

```
1  int exgcd(int a, int b, int &x, int &y) {  
2      if (b) {  
3          int d = exgcd(b, a % b, y, x);  
4          y -= a / b * x;  
5          return d;  
6      } else {  
7          x = 1, y = 0;  
8          return a;  
9      }  
10 }
```

容易归纳证明，当 a, b 均为非负整数时，有：

Theorem 2.5

设扩展 *Euclid* 算法求得的解为 $ax + by = d$ ，则 $|x| \leq b$ ，
 $|y| \leq a$ 。

容易归纳证明, 当 a, b 均为非负整数时, 有:

Theorem 2.5

设扩展 *Euclid* 算法求得的解为 $ax + by = d$, 则 $|x| \leq b$,
 $|y| \leq a$ 。

这也是扩展 *Euclid* 算法**不需要**使用更高一级整数类型的原因。

容易归纳证明，当 a, b 均为非负整数时，有：

Theorem 2.5

设扩展 *Euclid* 算法求得的解为 $ax + by = d$ ，则 $|x| \leq b$ ， $|y| \leq a$ 。

这也是扩展 *Euclid* 算法不需要使用更高一级整数类型的原因。

注意到该二元一次不定方程的所有解形如 $\left(x + \frac{b}{d}t, y - \frac{a}{d}t\right)$ ，

因此我们可以得到任意范围的解。

容易归纳证明, 当 a, b 均为非负整数时, 有:

Theorem 2.5

设扩展 *Euclid* 算法求得的解为 $ax + by = d$, 则 $|x| \leq b$, $|y| \leq a$ 。

这也是扩展 *Euclid* 算法**不需要**使用更高一级整数类型的原因。

注意到该二元一次不定方程的所有解形如 $\left(x + \frac{b}{d}t, y - \frac{a}{d}t\right)$,

因此我们可以得到任意范围的解。

当然, *Stein* 算法也可以改造成对应的扩展 *Stein* 算法。

Example 3

给定整数 a, b , 已知 $\sqrt{ab}$ 是整数, 求 $\sqrt{ab}$ 。

Example 3

给定整数 a, b , 已知 $\sqrt{ab}$ 是整数, 求 $\sqrt{ab}$ 。

Solution

设 $(a, b) = d$, 则 $a = du^2, b = dv^2$ 。

Example 3

给定整数 a, b , 已知 $\sqrt{ab}$ 是整数, 求 $\sqrt{ab}$ 。

Solution

设 $(a, b) = d$, 则 $a = du^2, b = dv^2$ 。

Example 4

给定整数 a, b, c , 已知 $\frac{ab}{c}$ 是整数, 求 $\frac{ab}{c}$ 。

Example 3

给定整数 a, b , 已知 $\sqrt{ab}$ 是整数, 求 $\sqrt{ab}$ 。

Solution

设 $(a, b) = d$, 则 $a = du^2, b = dv^2$ 。

Example 4

给定整数 a, b, c , 已知 $\frac{ab}{c}$ 是整数, 求 $\frac{ab}{c}$ 。

Solution

分步约分。

考虑 $m \in \mathbb{N}^*$ 。则考虑一切整数除以 m 所得的余数，一共有 m 种可能的情形： $0, 1, \dots, m-1$ 。我们将除以 m 余数为 r 的集合记为模 m 的 r 等价类，记为 $(\bar{r})_m$ 或 $\bar{r}$ ，即

$$(\bar{r})_m = \{k \cdot m + r \mid k \in \mathbb{Z}\}$$

考虑 $m \in \mathbb{N}^*$ 。则考虑一切整数除以 m 所得的余数，一共有 m 种可能的情形： $0, 1, \dots, m-1$ 。我们将除以 m 余数为 r 的集合记为模 m 的 r 等价类，记为 $(\bar{r})_m$ 或 $\bar{r}$ ，即

$$(\bar{r})_m = \{k \cdot m + r \mid k \in \mathbb{Z}\}$$

如果两个整数 a, b 在同一个等价类中，说明它们模 m 的余数相同，则我们称 a, b 关于模 m 同余，用 $a \equiv b \pmod{m}$ 表示。

考虑 $m \in \mathbb{N}^*$ 。则考虑一切整数除以 m 所得的余数，一共有 m 种可能的情形： $0, 1, \dots, m-1$ 。我们将除以 m 余数为 r 的集合记为模 m 的 r 等价类，记为 $(\bar{r})_m$ 或 $\bar{r}$ ，即

$$(\bar{r})_m = \{k \cdot m + r \mid k \in \mathbb{Z}\}$$

如果两个整数 a, b 在同一个等价类中，说明它们模 m 的余数相同，则我们称 a, b 关于模 m 同余，用 $a \equiv b \pmod{m}$ 表示。

Corollary 3.1

同余具有自反性、对称性和传递性，是一个等价关系。

Definition 3.1 (Complete residue system)

对正整数 m ，在 m 个等价类中各取一个元素，构成的集合称为模 m 的一个完全剩余系，简称完系，记作 CRS 。

Definition 3.1 (Complete residue system)

对正整数 m , 在 m 个等价类中各取一个元素, 构成的集合称为模 m 的一个完全剩余系, 简称完系, 记作 CRS 。

特别地, $\{0, 1, \dots, m-1\}$ 称为模 m 的最小非负完系 (LRS), 以奇数为例, $\left\{-\frac{m-1}{2}, \dots, \frac{m-1}{2}\right\}$ 称为模 m 的绝对最小完系 (ARS)。

Definition 3.1 (Complete residue system)

对正整数 m ，在 m 个等价类中各取一个元素，构成的集合称为模 m 的一个完全剩余系，简称完系，记作 CRS。

特别地， $\{0, 1, \dots, m-1\}$ 称为模 m 的最小非负完系 (LRS)，以奇数为例， $\left\{-\frac{m-1}{2}, \dots, \frac{m-1}{2}\right\}$ 称为模 m 的绝对最小完系 (ARS)。

如，`unsigned int` 类型和 `signed int` 类型分别对应模 2^{32} 的最小非负完系和绝对最小完系。

Definition 3.2 (Primitive equivalence class)

等价类 $(\bar{r})_m$ 称为本原的, 如果 r 与 m 互素。

Definition 3.2 (Primitive equivalence class)

等价类 $(\bar{r})_m$ 称为本原的, 如果 r 与 m 互素。

Definition 3.3 (Reduced residue system)

对正整数 m , 在其所有本原等价类中各取一个元素, 构成的集合称为模 m 的一个简化剩余系, 简称缩系, 记作 RRS 。

Definition 3.2 (Primitive equivalence class)

等价类 $(\bar{r})_m$ 称为本原的, 如果 r 与 m 互素。

Definition 3.3 (Reduced residue system)

对正整数 m , 在其所有本原等价类中各取一个元素, 构成的集合称为模 m 的一个简化剩余系, 简称缩系, 记作 RRS 。

模 m 的本原等价类数目, 称为 m 的 **Euler 函数**, 记作 $\varphi(m)$ 。它也等于 $1 \sim m$ 中与 m 互素的数的个数。

可以发现，对于两个等价类 $\bar{a}, \bar{b}$ ，它们的加、减、乘法都是良定义的，即无论取等价类中的哪个元素，最终得到的结果都在一个确定的等价类中。

可以发现，对于两个等价类 $\bar{a}, \bar{b}$ ，它们的加、减、乘法都是良定义的，即无论取等价类中的哪个元素，最终得到的结果都在一个确定的等价类中。

因此我们可以在等价类中进行加、减、乘法。为了方便，通常取最小非负完系中的元素进行运算，这就是所谓的模 m 算术 (用 $\mathbb{Z}/m\mathbb{Z}$ 或 Z_m 表示)。

而一般情况下，等价类中作除法可能会有多个解，也有可能无解。

而一般情况下，等价类中作除法可能会有多个解，也有可能无解。

事实上，考虑模 m 意义下的 $a \div b$ ，其实就是寻找整数 r 使得 $b \cdot r \equiv a \pmod{m}$ 。

而一般情况下，等价类中作除法可能会有多个解，也有可能无解。

事实上，考虑模 m 意义下的 $a \div b$ ，其实就是寻找整数 r 使得 $b \cdot r \equiv a \pmod{m}$ 。

因此，由等价类的性质知存在整数 ξ 满足 $b \cdot r + m \cdot \xi = a$ ，即以 (r, ξ) 为元的二元一次不定方程有解。

而一般情况下，等价类中作除法可能会有多个解，也有可能无解。

事实上，考虑模 m 意义下的 $a \div b$ ，其实就是寻找整数 r 使得 $b \cdot r \equiv a \pmod{m}$ 。

因此，由等价类的性质知存在整数 ξ 满足 $b \cdot r + m \cdot \xi = a$ ，即以 (r, ξ) 为元的二元一次不定方程有解。

这要求 $\gcd(b, m) \mid a$ 。为了适应对所有 a 都可以的情形，我们希望 $\gcd(b, m) = 1$ 。此时，方程 $b \cdot r + m \cdot \xi = 1$ 有解 (r, ξ) ，亦即存在整数 r 满足 $b \cdot r \equiv 1 \pmod{m}$ 。

而一般情况下，等价类中作除法可能会有多个解，也有可能无解。

事实上，考虑模 m 意义下的 $a \div b$ ，其实就是寻找整数 r 使得 $b \cdot r \equiv a \pmod{m}$ 。

因此，由等价类的性质知存在整数 ξ 满足 $b \cdot r + m \cdot \xi = a$ ，即以 (r, ξ) 为元的二元一次不定方程有解。

这要求 $\gcd(b, m) \mid a$ 。为了适应对所有 a 都可以的情形，我们希望 $\gcd(b, m) = 1$ 。此时，方程 $b \cdot r + m \cdot \xi = 1$ 有解 (r, ξ) ，亦即存在整数 r 满足 $b \cdot r \equiv 1 \pmod{m}$ 。

我们将满足 $b \cdot r \equiv 1 \pmod{m}$ 的整数 r ，称为 b 在模 m 下的逆 (逆元，乘法逆，数论倒数)，记为 $b^{-1} \pmod{m}$ 。

我们在小学二年级就学过，“除以一个数，等于乘上那个数的倒数”。逆扮演的角色就是倒数，因此也称之为数论倒数。

我们在小学二年级就学过，“除以一个数，等于乘上那个数的倒数”。逆扮演的角色就是倒数，因此也称之为数论倒数。

令 $r \equiv b^{-1} \cdot a \pmod{m}$ ，则 $b \cdot r \equiv b \cdot b^{-1} \cdot a \equiv a \pmod{m}$ ，因此 r 就是模 m 意义下的 $a \div b$ 。

我们在小学二年级就学过，“除以一个数，等于乘上那个数的倒数”。逆扮演的角色就是倒数，因此也称之为数论倒数。

令 $r \equiv b^{-1} \cdot a \pmod{m}$ ，则 $b \cdot r \equiv b \cdot b^{-1} \cdot a \equiv a \pmod{m}$ ，因此 r 就是模 m 意义下的 $a \div b$ 。

同时，当 $(b, m) = 1$ 时，这样的 r 在等价类意义下是唯一的，从而此时可以做模意义下的除法。

我们在小学二年级就学过，“除以一个数，等于乘上那个数的倒数”。逆扮演的角色就是倒数，因此也称之为数论倒数。

令 $r \equiv b^{-1} \cdot a \pmod{m}$ ，则 $b \cdot r \equiv b \cdot b^{-1} \cdot a \equiv a \pmod{m}$ ，因此 r 就是模 m 意义下的 $a \div b$ 。

同时，当 $(b, m) = 1$ 时，这样的 r 在等价类意义下是唯一的，从而此时可以做模意义下的除法。

而上述过程也告诉我们求 b^{-1} 的方法——使用扩展 Euclid 算法，解二元一次不定方程 $b \cdot r + m \cdot \xi = 1$ 。

我们在小学二年级就学过，“除以一个数，等于乘上那个数的倒数”。逆扮演的角色就是倒数，因此也称之为数论倒数。

令 $r \equiv b^{-1} \cdot a \pmod{m}$ ，则 $b \cdot r \equiv b \cdot b^{-1} \cdot a \equiv a \pmod{m}$ ，因此 r 就是模 m 意义下的 $a \div b$ 。

同时，当 $(b, m) = 1$ 时，这样的 r 在等价类意义下是唯一的，从而此时可以做模意义下的除法。

而上述过程也告诉我们求 b^{-1} 的方法——使用扩展 Euclid 算法，解二元一次不定方程 $b \cdot r + m \cdot \xi = 1$ 。

后面我们还会介绍三种求逆的方法，在学习了相关理论后会提及。

我们在小学二年级就学过，“除以一个数，等于乘上那个数的倒数”。逆扮演的角色就是倒数，因此也称之为数论倒数。

令 $r \equiv b^{-1} \cdot a \pmod{m}$ ，则 $b \cdot r \equiv b \cdot b^{-1} \cdot a \equiv a \pmod{m}$ ，因此 r 就是模 m 意义下的 $a \div b$ 。

同时，当 $(b, m) = 1$ 时，这样的 r 在等价类意义下是唯一的，从而此时可以做模意义下的除法。

而上述过程也告诉我们求 b^{-1} 的方法——使用扩展 Euclid 算法，解二元一次不定方程 $b \cdot r + m \cdot \xi = 1$ 。

后面我们还会介绍三种求逆的方法，在学习了相关理论后会提及。

和实数的幂一样，对于与模互素的整数 b ，我们可以类似的定义它的负指数幂： $b^{-r} \equiv (b^{-1})^r \equiv (b^r)^{-1} \pmod{m}$ 。

Example 5 (RSA)

有两个大素数 p, q , 满足 $p \equiv q \equiv 2 \pmod{3}$, 记 $n = pq$ 。

现在有一个整数 $m \in Z_n$, 给定 $n, m^3 \bmod n, (m+1)^3 \bmod n$ 的值, 求 m 的所有可能值。

Example 5 (RSA)

有两个大素数 p, q , 满足 $p \equiv q \equiv 2 \pmod{3}$, 记 $n = pq$ 。

现在有一个整数 $m \in Z_n$, 给定 $n, m^3 \bmod n, (m+1)^3 \bmod n$ 的值, 求 m 的所有可能值。

(注: p, q 足够大以至于你无法通过 n 得到 p 和 q 。同时, p, q 的大小可以使你忽略高精度计算所带来的代价 (可以使用 *Python* 完成)。)

(前置: $p \equiv q \equiv 2 \pmod{3}$ 的用途是, 对于一切整数 x , 均有 $(x^2 + 3, pq) = 1$ 。)

Example 5 (RSA)

有两个大素数 p, q , 满足 $p \equiv q \equiv 2 \pmod{3}$, 记 $n = pq$ 。

现在有一个整数 $m \in Z_n$, 给定 $n, m^3 \bmod n, (m+1)^3 \bmod n$ 的值, 求 m 的所有可能值。

(注: p, q 足够大以至于你无法通过 n 得到 p 和 q 。同时, p, q 的大小可以使你忽略高精度计算所带来的代价 (可以使用 *Python* 完成)。)

(前置: $p \equiv q \equiv 2 \pmod{3}$ 的用途是, 对于一切整数 x , 均有 $(x^2 + 3, pq) = 1$ 。)

Solution

$$m = \frac{(m+1)^3 + 2m^3 - 1}{(m+1)^3 - m^3 + 2}$$

Example 5 (RSA)

有两个大素数 p, q , 满足 $p \equiv q \equiv 2 \pmod{3}$, 记 $n = pq$.

现在有一个整数 $m \in \mathbb{Z}_n$, 给定 $n, m^3 \bmod n, (m+1)^3 \bmod n$ 的值, 求 m 的所有可能值。

(注: p, q 足够大以至于你无法通过 n 得到 p 和 q 。同时, p, q 的大小可以使你忽略高精度计算所带来的代价 (可以使用 *Python* 完成)。)

(前置: $p \equiv q \equiv 2 \pmod{3}$ 的用途是, 对于一切整数 x , 均有 $(x^2 + 3, pq) = 1$ 。)

Solution

$$m = \frac{(m+1)^3 + 2m^3 - 1}{(m+1)^3 - m^3 + 2}$$

注意分母 $(m+1)^3 - m^3 + 2 = 3(m^2 + m + 1)$, 显然 $(3, n) = 1$, 而对于 $m^2 + m + 1$, 有 $(4(m^2 + m + 1), n) = ((2m+1)^2 + 3, n) = 1$, 即分母存在逆元。显然 m 无解或唯一, 别忘了验证。

素数是数论中最重要的数，定义如下：

Definition 1.1 (Prime number & Composite number)

对于一个大于 1 的整数 n ，如果 $\mathcal{D}(n) = \{1, n\}$ ，则 n 为素数 (质数)，否则 n 为合数。用 $\mathbb{P}$ 表示所有素数构成的集合。

素数是数论中最重要的数，定义如下：

Definition 1.1 (Prime number & Composite number)

对于一个大于 1 的整数 n ，如果 $\mathcal{D}(n) = \{1, n\}$ ，则 n 为素数 (质数)，否则 n 为合数。用 $\mathbb{P}$ 表示所有素数构成的集合。

注意到 $\{1, n\} \subseteq \mathcal{D}(n)$ ，因此也可以说素数是没有非平凡因子的数。

素数是数论中最重要的数，定义如下：

Definition 1.1 (Prime number & Composite number)

对于一个大于 1 的整数 n ，如果 $\mathcal{D}(n) = \{1, n\}$ ，则 n 为素数 (质数)，否则 n 为合数。用 $\mathbb{P}$ 表示所有素数构成的集合。

注意到 $\{1, n\} \subseteq \mathcal{D}(n)$ ，因此也可以说素数是没有非平凡因子的数。

素数的一个较为重要的性质如下：

Theorem 1.1

对于 $p \in \mathbb{P}$ 和整数 n ， $(p, n) = 1$ 与 $p \mid n$ 成立且恰成立一个。

Theorem 1.2 (Fundamental theorem of arithmetic)

每一个大于 1 的整数 n 都能表示成如下形式：

$$n = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_l^{\alpha_l} \quad (1)$$

其中 $p_1 < p_2 < \cdots < p_l$ 是素数, $l, \alpha_1, \alpha_2, \cdots, \alpha_l$ 是正整数。

Theorem 1.2 (Fundamental theorem of arithmetic)

每一个大于 1 的整数 n 都能表示成如下形式：

$$n = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_l^{\alpha_l} \quad (1)$$

其中 $p_1 < p_2 < \cdots < p_l$ 是素数， $l, \alpha_1, \alpha_2, \cdots, \alpha_l$ 是正整数。

(1) 式被称为 n 的**标准分解式**。将 n 写成 (1) 式的过程被称为**分解质因数** (Prime factorization)。

Theorem 1.3 (Fermat's little theorem)

对于素数 p 和整数 a , 有 $a^p \equiv a \pmod{p}$ 。

Theorem 1.3 (Fermat's little theorem)

对于素数 p 和整数 a , 有 $a^p \equiv a \pmod{p}$ 。

Corollary 1.1

对于素数 p , 若 $(a, p) = 1$, 则 $a^{p-1} \equiv 1 \pmod{p}$, $a^{p-2} \equiv a^{-1} \pmod{p}$ 。

Theorem 1.3 (Fermat's little theorem)

对于素数 p 和整数 a , 有 $a^p \equiv a \pmod{p}$ 。

Corollary 1.1

对于素数 p , 若 $(a, p) = 1$, 则 $a^{p-1} \equiv 1 \pmod{p}$, $a^{p-2} \equiv a^{-1} \pmod{p}$ 。

推论 1.1 给了我们第二种求逆的方法: 对于模素数的逆, 可以使用快速幂转化为求 $p-2$ 次方。

第三种求逆的方法是线性求逆，它可以在关于 n 的线性时间内求出 $1 \sim n$ 中所有整数在 $(\text{mod } p)$ 下的逆，这里要求 $n < p$ 。

第三种求逆的方法是线性求逆，它可以在关于 n 的线性时间内求出 $1 \sim n$ 中所有整数在 $(\text{mod } p)$ 下的逆，这里要求 $n < p$ 。它的思想和第二归纳法较为类似：

第三种求逆的方法是线性求逆，它可以在关于 n 的线性时间内求出 $1 \sim n$ 中所有整数在 $(\text{mod } p)$ 下的逆，这里要求 $n < p$ 。

它的思想和第二归纳法较为类似：

假设我们已经求出了 $1 \sim n-1$ 在 $(\text{mod } p)$ 下的逆，考虑求 n^{-1} 。由带余除法，设 $p \div n = q \dots r$ (显然 $r \neq 0$)，则

$$p = n \cdot q + r$$

$$0 \equiv n \cdot q + r \pmod{p}$$

$$0 \equiv q + r \cdot n^{-1} \pmod{p}$$

$$0 \equiv q \cdot r^{-1} + n^{-1} \pmod{p}$$

$$n^{-1} \equiv -q \cdot r^{-1} \pmod{p}$$

第三种求逆的方法是线性求逆，它可以在关于 n 的线性时间内求出 $1 \sim n$ 中所有整数在 $(\text{mod } p)$ 下的逆，这里要求 $n < p$ 。

它的思想和第二归纳法较为类似：

假设我们已经求出了 $1 \sim n-1$ 在 $(\text{mod } p)$ 下的逆，考虑求 n^{-1} 。由带余除法，设 $p \div n = q \dots r$ (显然 $r \neq 0$)，则

$$p = n \cdot q + r$$

$$0 \equiv n \cdot q + r \pmod{p}$$

$$0 \equiv q + r \cdot n^{-1} \pmod{p}$$

$$0 \equiv q \cdot r^{-1} + n^{-1} \pmod{p}$$

$$n^{-1} \equiv -q \cdot r^{-1} \pmod{p}$$

而 $1 \leq r \leq n-1$ ，因此 r^{-1} 是已知的，我们完成了整个过程。

该算法也因代码实现简单而被广泛使用：

```
1 inv[1] = 1;
2 for (int i = 2; i <= n; ++i) {
3     inv[i] = (long long)(p - p / i) * inv[p % i] % p;
4 }
```

该算法也因代码实现简单而被广泛使用：

```
1 inv[1] = 1;  
2 for (int i = 2; i <= n; ++i) {  
3     inv[i] = (long long)(p - p / i) * inv[p % i] % p;  
4 }
```

同时，这个思想为打表处理离散对数也有较大帮助。

不过，频繁地访问 `inv[p % i]` 对缓存并不是很友好。尽管可以使用整除分块进行略微优化，下面介绍一种更加通用的算法。

不过，频繁地访问 `inv[p % i]` 对缓存并不是很友好。尽管可以使用整除分块进行略微优化，下面介绍一种更加通用的算法。

Example 6

给定 m 和 n 个与 m 互素的正整数 $a_1, a_2, \dots, a_n$ ，你需要求出 $a_1^{-1}, a_2^{-1}, \dots, a_n^{-1}$ 。

不过，频繁地访问 `inv[p % i]` 对缓存并不是很友好。尽管可以使用整除分块进行略微优化，下面介绍一种更加通用的算法。

Example 6

给定 m 和 n 个与 m 互素的正整数 $a_1, a_2, \dots, a_n$ ，你需要求出 $a_1^{-1}, a_2^{-1}, \dots, a_n^{-1}$ 。

Solution

显然有一个 $O(n \log m)$ 的暴力，使用扩展 *Euclid* 算法解决。

不过，频繁地访问 `inv[p % i]` 对缓存并不是很友好。尽管可以使用整除分块进行略微优化，下面介绍一种更加通用的算法。

Example 6

给定 m 和 n 个与 m 互素的正整数 $a_1, a_2, \dots, a_n$ ，你需要求出 $a_1^{-1}, a_2^{-1}, \dots, a_n^{-1}$ 。

Solution

显然有一个 $O(n \log m)$ 的暴力，使用扩展 *Euclid* 算法解决。
考虑 $n = 2$ 时的做法，我们有 $x^{-1} = y \cdot (xy)^{-1}$, $y^{-1} = x \cdot (xy)^{-1}$ ，从而只需要求一次逆。

不过，频繁地访问 `inv[p % i]` 对缓存并不是很友好。尽管可以使用整除分块进行略微优化，下面介绍一种更加通用的算法。

Example 6

给定 m 和 n 个与 m 互素的正整数 $a_1, a_2, \dots, a_n$ ，你需要求出 $a_1^{-1}, a_2^{-1}, \dots, a_n^{-1}$ 。

Solution

显然有一个 $O(n \log m)$ 的暴力，使用扩展 *Euclid* 算法解决。考虑 $n = 2$ 时的做法，我们有 $x^{-1} = y \cdot (xy)^{-1}$, $y^{-1} = x \cdot (xy)^{-1}$ ，从而只需要求一次逆。很容易就可以将这个算法扩展到一般 n 的情形。

Solution

令 $P \equiv a_1 a_2 \cdots a_n \pmod{m}$, 则 $(P, m) = 1$, 于是 P 存在逆。

Solution

令 $P \equiv a_1 a_2 \cdots a_n \pmod{m}$, 则 $(P, m) = 1$, 于是 P 存在逆。

此时, 有 $a_i = \frac{(a_1 a_2 \cdots a_{i-1})(a_{i+1} a_{i+2} \cdots a_n)}{a_1 a_2 \cdots a_n} \equiv (a_1 a_2 \cdots a_{i-1})$
 $(a_{i+1} a_{i+2} \cdots a_n) \cdot P^{-1} \pmod{m}$ 。

Solution

令 $P \equiv a_1 a_2 \cdots a_n \pmod{m}$, 则 $(P, m) = 1$, 于是 P 存在逆。

此时, 有 $a_i = \frac{(a_1 a_2 \cdots a_{i-1})(a_{i+1} a_{i+2} \cdots a_n)}{a_1 a_2 \cdots a_n} \equiv (a_1 a_2 \cdots a_{i-1})$

$(a_{i+1} a_{i+2} \cdots a_n) \cdot P^{-1} \pmod{m}$ 。

故只需维护前缀积、后缀积即可, 时间复杂度 $O(n + \log m)$ 。
该算法在 *Lagrange* 插值时也有较大用途。

有的时候，整个问题是在线的——即你不能立即知道所有数，然后一块儿求逆元，必须一个一个求。

有的时候，整个问题是在线的——即你不能立即知道所有数，然后一块儿求逆元，必须一个一个求。

这个时候，有一种方法被称之为惰性求值，具体的思想是用类似“分数”的结构去维护。它也去除了所谓的 $\log$ ，但是代价是一点常数和两倍内存。

有的时候，整个问题是在线的——即你不能立即知道所有数，然后一块儿求逆元，必须一个一个求。

这个时候，有一种方法被称之为惰性求值，具体的思想是用类似“分数”的结构去维护。它也去除了所谓的 $\log$ ，但是代价是一点常数和两倍内存。

对于模 m ，我们用符号 $\frac{a}{b}$ 来表示 $a \cdot b^{-1} \pmod{m}$ ，其中 $(b, m) = 1$ 。则，这样的数是可以进行加减乘除的：

有的时候，整个问题是在线的——即你不能立即知道所有数，然后一块儿求逆元，必须一个一个求。

这个时候，有一种方法被称之为惰性求值，具体的思想是用类似“分数”的结构去维护。它也去除了所谓的 $\log$ ，但是代价是一点常数和两倍内存。

对于模 m ，我们用符号 $\frac{a}{b}$ 来表示 $a \cdot b^{-1} \pmod{m}$ ，其中 $(b, m) = 1$ 。则，这样的数是可以进行加减乘除的：

$$\blacksquare \quad \frac{a}{b} \pm \frac{c}{d} = \frac{ad \pm bc}{bd}。$$

有的时候，整个问题是在线的——即你不能立即知道所有数，然后一块儿求逆元，必须一个一个求。

这个时候，有一种方法被称之为惰性求值，具体的思想是用类似“分数”的结构去维护。它也去除了所谓的 $\log$ ，但是代价是一点常数和两倍内存。

对于模 m ，我们用符号 $\frac{a}{b}$ 来表示 $a \cdot b^{-1} \pmod{m}$ ，其中 $(b, m) = 1$ 。则，这样的数是可以进行加减乘除的：

- $\frac{a}{b} \pm \frac{c}{d} = \frac{ad \pm bc}{bd}。$
- $\frac{a}{b} \cdot \frac{c}{d} = \frac{ac}{bd}。$

有的时候，整个问题是在线的——即你不能立即知道所有数，然后一块儿求逆元，必须一个一个求。

这个时候，有一种方法被称之为惰性求值，具体的思想是用类似“分数”的结构去维护。它也去除了所谓的 $\log$ ，但是代价是一点常数和两倍内存。

对于模 m ，我们用符号 $\frac{a}{b}$ 来表示 $a \cdot b^{-1} \pmod{m}$ ，其中 $(b, m) = 1$ 。则，这样的数是可以进行加减乘除的：

■ $\frac{a}{b} \pm \frac{c}{d} = \frac{ad \pm bc}{bd}。$

■ $\frac{a}{b} \cdot \frac{c}{d} = \frac{ac}{bd}。$

■ 若 $(c, m) = 1$ ，则 $\frac{a}{b} / \frac{c}{d} = \frac{ad}{bc}。$

有的时候，整个问题是在线的——即你不能立即知道所有数，然后一块儿求逆元，必须一个一个求。

这个时候，有一种方法被称之为惰性求值，具体的思想是用类似“分数”的结构去维护。它也去除了所谓的 $\log$ ，但是代价是一点常数和两倍内存。

对于模 m ，我们用符号 $\frac{a}{b}$ 来表示 $a \cdot b^{-1} \pmod{m}$ ，其中 $(b, m) = 1$ 。则，这样的数是可以进行加减乘除的：

$$\blacksquare \quad \frac{a}{b} \pm \frac{c}{d} = \frac{ad \pm bc}{bd}。$$

$$\blacksquare \quad \frac{a}{b} \cdot \frac{c}{d} = \frac{ac}{bd}。$$

$$\blacksquare \quad \text{若 } (c, m) = 1, \text{ 则 } \frac{a}{b} / \frac{c}{d} = \frac{ad}{bc}。$$

当最后要求输出时，再求一遍分母的逆，如果输出不是在线，那么也可以用 Example 4 所介绍的方法解决。

Example 7 (水题)

维护 n 个数 $a_1, a_2, \dots, a_n$, 支持 q 次操作, 操作分为两种:

- 给定 l, r , 对于每个 $i \in [l, r]$, 令 $a_i \leftarrow a_i + x$ 。
- 给定 l, r , 对于每个 $i \in [l, r]$, 令 $a_i \leftarrow a_i^{P-2}$ 。

你需要在每次操作后, 输出 $\left(\sum_{i=1}^n a_i\right) \bmod P$ 的值。其中 P 为不超过 10^{18} 的奇素数。

$n, q \leq 2000$ 。

Example 7 (水题)

维护 n 个数 $a_1, a_2, \dots, a_n$, 支持 q 次操作, 操作分为两种:

- 给定 l, r , 对于每个 $i \in [l, r]$, 令 $a_i \leftarrow a_i + x$ 。
- 给定 l, r , 对于每个 $i \in [l, r]$, 令 $a_i \leftarrow a_i^{P-2}$ 。

你需要在每次操作后, 输出 $\left(\sum_{i=1}^n a_i\right) \bmod P$ 的值。其中 P 为不超过 10^{18} 的奇素数。
 $n, q \leq 2000$ 。

Solution

使用惰性求值, 时间复杂度 $O(nq + q \log P)$ 。